

Universal Embedding Function for Traffic Classification via QUIC Domain Recognition Pretraining: A Transfer Learning Success

Jan Luxemburk^{id}, Karel Hynek^{id}, Richard Plný^{id}, and Tomáš Čejka^{id}

Abstract—Encrypted traffic classification (TC) methods must adapt to new protocols and extensions as well as to advancements in other machine learning fields. In this paper, we adopt a transfer learning setup best known from computer vision. We first pretrain an embedding model on a complex task with a large number of classes and then transfer it to seven established TC datasets. The pretraining task is recognition of SNI domains in encrypted QUIC traffic, which in itself is a challenge for network monitoring due to the growing adoption of TLS Encrypted Client Hello. Our training pipeline—featuring a disjoint class setup, ArcFace loss function, and a modern deep learning architecture—aims to produce universal embeddings applicable across tasks. A transfer method based on model fine-tuning surpassed SOTA performance on nine of ten downstream TC tasks, with an average improvement of 6.4%. Furthermore, a comparison with a baseline method using raw packet sequences revealed unexpected findings with potential implications for the broader TC field. We released the model architecture, trained weights, and codebase for transfer learning experiments.

Index Terms—Traffic classification, transfer learning, deep learning, encrypted traffic, QUIC.

I. INTRODUCTION

IN THIS paper, we propose a universal embedding (mapping) function that transforms packet sequences into an embedding vector space. The core idea is to map similar packet sequences close to each other in the embedding space while keeping dissimilar ones far apart. The embedding function serves as a feature extractor, enabling a nearest neighbors (k-NN) classifier to make predictions. As our focus is on encrypted traffic, we utilize the standard input representation unaffected by encryption: packet size, direction, and inter-packet time of the first N packets. We deliberately avoid using payload as a model input due to its limited value in analyzing initial handshakes of encrypted protocols, a task we consider

better suited to protocol dissectors and pattern-matching. Additionally, in deployment scenarios where inference is not performed directly on monitoring probes, transmitting payload data introduces both privacy and performance issues.

Building on our previous research in fine-grained traffic classification (TC) for TLS [1] and QUIC [2], we design and train the embedding function using the CESNET-QUIC22 [3] dataset. This dataset includes Server Name Indication (SNI) domains as labels, enabling a classification task focused on inferring exact domain names from packet sequences. This domain recognition task serves two important roles in this paper. First, the growing adoption of the Encrypted Client Hello (ECH) extension has made domain recognition increasingly important. In combination with TLS 1.3, ECH removes all plaintext handshake fields that have traditionally enabled visibility into encrypted traffic. Our solution, which relies solely on packet sequences and is unaffected by ECH, is able to infer the correct domain name in 94.83% of cases, even when evaluated on test domains entirely disjoint from the embedding function's training set. Second, domain recognition is well suited as a pretraining task due to its complexity, large number of classes, and straightforward labeling process. Transfer learning leverages models trained on one task to adapt them for a different but related task, under the assumption that certain knowledge (i.e., extracted features, learned traffic patterns and characteristics) is shared and transferable. This approach is widely used in computer vision and natural language processing, where the typical experimental pipeline involves fine-tuning large models pretrained on public datasets.

To evaluate the transfer learning approach, we tested our pretrained embedding function on seven datasets: ISCXVPN2016 [4], MIRAGE19 [5], MIRAGE22 [6], UTMOBILENET21 [7], UCDAVIS19 [8], CESNET-TLS22 [1], and AppClassNet [9], presenting ten downstream TC tasks in total. We evaluated three transfer techniques: (a) using fixed pretrained embeddings with a k-NN classifier, (b) using fixed pretrained embeddings with a linear classifier (also known as linear probing), and (c) fine-tuning the embedding function on each downstream task. To isolate the contribution of the transfer learning, we also compared against training the same model from scratch on the downstream tasks.

The results are promising: the fine-tuning approach proved to be the best, surpassing state-of-the-art (SOTA) performance on nine downstream tasks and outperforming training from

Received 18 February 2025; revised 14 August 2025 and 6 November 2025; accepted 6 December 2025. Date of publication 11 December 2025; date of current version 13 January 2026. This research was funded by the Ministry of Interior of the Czech Republic, grant No. VJ02010024: “Flow-Based Encrypted Traffic Analysis” and also supported by the Grant Agency of the CTU in Prague, grant No. SGS23/207/OHK3/3T/18. Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic. The associate editor coordinating the review of this article and approving it for publication was T. Inoue. (Corresponding author: Jan Luxemburk.)

Jan Luxemburk, Karel Hynek, and Richard Plný are with the Faculty of Information Technology, Czech Technical University in Prague, Prague 160 00, Czech Republic, and also with the CESNET Association, Prague 160 00, Czech Republic (e-mail: luxemburk@cesnet.cz).

Tomáš Čejka is with CESNET Association, Prague 160 00, Czech Republic. Digital Object Identifier 10.1109/TNSM.2025.3642984

scratch on eight. A k-NN classifier in the learned embedding space also exceeded SOTA performance, although the gains were more modest compared to fine-tuning. This offers a viable alternative when fine-tuning or training from scratch is not feasible, for example due to insufficient labeled data for the downstream task. Finally, our experiments revealed an intriguing finding: a k-NN classifier using L1 distance on the first 10 packet features—referred to as the input-space baseline—also performed quite well across all datasets.

Main contributions

(i) We designed and developed **an embedding function for packet sequences to serve as a foundation for classifiers of encrypted network traffic**. It is based on a neural network architecture 30pktTCNET_256 that combines existing, well-proven techniques, such as ResNet-like convolutional blocks, packet feature embeddings, and Generalized Mean Pooling (GeM) [10]. For initializing packet feature embeddings, we adopted Piecewise Linear Encoding (PLE) [11], a method that outperforms random initialization and has not previously been used in the TC domain. Although the architecture is largely based on established techniques—with the exception of PLE—we rigorously tuned all its components to uncover the true limits of CNN-based processing of packet sequences.

(ii) We present **a domain recognition solution that enables domain-level visibility into encrypted traffic, even in the presence of TLS 1.3 and ECH**. Using a k-NN classifier in the embedding space, our method achieves 94.83% accuracy and 79.35% recall under a class-disjoint evaluation setup. We further show that domain recognition serves as an effective pretraining task, as the learned embeddings generalize across ten downstream TC tasks.

(iii) With the fine-tuning transfer method, we **surpass the SOTA performance on nine of the ten downstream TC tasks, achieving a remarkable average improvement of 6.4%**. Compared to training from scratch, fine-tuning pretrained weights provides a 2.1% average gain, empirically confirming the benefits of transfer learning.

The paper is organized as follows: Section II provides a review of related research. Section III describes the experimental setup used to develop the embedding function, covering the dataset, data preparation, training loop, loss function, and deep learning (DL) architecture. Section IV presents the results of the domain recognition task, along with ablation studies examining parts of the solution. Section V describes the transfer of the trained embedding function to seven additional TC datasets and discusses the achieved results. Section VI addresses limitations, and Section VII concludes the paper by summarizing key contributions and outlining future directions.

II. RELATED WORK

We begin by outlining how transfer learning has been applied within the TC domain. Next, we review influential papers on representation learning for TC, highlighting methodological similarities to our approach and key differences. We deliberately omit detailed discussion of studies that rely heavily on payload data, focusing instead on methods that operate on packet sequences or on representations derived from them.

A. Transfer Learning in Traffic Classification

The objective of transfer learning is to capitalize on the knowledge encoded in a model M_0 , learned while solving a task T_0 , and adapt it to a different target task T_{target} . The key intuition is that the closer the relationship between T_0 and T_{target} , the more effectively the learned representations (i.e., features) can be reused and transferred. Depending on the application, the label spaces of T_0 and T_{target} may be disjoint (e.g., when pretraining on a pretext task), partially overlapping (e.g., when adapting to new classes), or identical (e.g., when adapting to a new operational environment). Transfer learning has gained significant traction in network classification and is applied in various scenarios discussed below.

1) *Cross-Dataset: Pretraining on pretext tasks before fine-tuning for downstream objectives*. ET-BERT [12] and YaTC [13] exemplify this strategy, leveraging large unlabeled datasets to pretrain transformer models in an NLP-like fashion using tokenization and masking. However, both operate on packet payloads and are thus outside the scope of detailed comparison in this work. Rezaei et al. [14] pretrained a 1D-CNN on QUIC packet sequences via a pretext task targeting flow bandwidth and duration prediction, and subsequently transferred it to Web service classification. This pretraining improved performance relative to direct training on Web services; nevertheless, jointly training all three tasks—bandwidth, duration, and Web service—yielded the strongest performance.

2) *New-Class Adaptation: Transferring knowledge from frequent to rare classes in incremental learning and few-shot learning (FSL)*. When the main constraint is a limited number of samples, transfer learning can serve as an alternative to, or a component within, various FSL approaches. Bovenzi et al. [15] studied IoT attack detection across four datasets and found that FSL methods outperform transfer learning. However, subsequent works by Di Monda et al. [16] and Monda et al. [17], focusing on intrusion detection and fast adaptation to new attacks, reported the opposite result: transfer learning methods were superior. In particular, the *Rethinking Few-Shot* method of Tian et al. [18] achieved excellent performance using a base model trained in a standard supervised fashion, with additional self-distillation to enhance the learned representations. The *Rethinking Few-Shot* method highlights that a strong base model can offer performance comparable to, or better than, standard FSL meta-learning approaches while enabling a simpler and arguably more straightforward adaptation to new classes. Tong et al. [19] proposed a feature-alignment approach for encrypted traffic classification. In their problem setup, the source domain provides abundant labeled data, whereas the target domain has many samples but few or no labels and includes additional classes. Their method introduces an auxiliary optimization objective, termed the smooth characteristic function, which encourages the feature distributions of the two domains to align during training. This alignment facilitates classification of previously unseen classes in the target domain, although the approach requires target-domain data to be available during base-model training.

3) *Cross-Network: Adapting models trained in one environment to operate in another*. Unlike most approaches

discussed so far, which aim to learn embeddings of individual network connections, Gioacchini et al. [20] learn embeddings of hosts. They employ DarkVec [21], a Word2Vec-based method that learns host embeddings from co-occurrence patterns—for example, when hosts contact the same server ports at the same time. The authors then study two transfer settings: (a) adapting host embeddings from a provider network to different, potentially label-scarce client networks, and (b) transferring knowledge learned from honeypots to scanners observed by a network telescope.

B. Contrastive Learning & Augmentations

Guarino et al. [22] focused on the task of finding better representations for TC that generalize across tasks, which is in line with the goals we set up for this work. The authors compared transfer-, meta-, and contrastive-learning approaches on MIRAGE19 and AppClassNet datasets. Both datasets were partitioned into training, validation, and test sets with disjoint classes, similar to how we partition domain names for the domain recognition task. However, their partitioning approach was based on class frequencies: the training set contained the most frequent classes, while the test set included the least frequent ones. In contrast, we used random partitioning independent of class frequencies. One of the findings of Guarino et al. is that supervised contrastive learning produces the best representations overall. Our work supports this finding, as there are notable similarities between the loss function used in our work, ArcFace, and the contrastive loss function used by Guarino et al.: SupCon [23] extended with class centers and cosine distance. Both loss functions operate by pulling similar samples toward their class centers using cosine similarity. The key distinction is that ArcFace introduces angular margins, which enhance class separation in the embedding space.

Wang et al. [24] presented a benchmark of data augmentations, evaluating 18 of them on three datasets: MIRAGE19, MIRAGE22, and a private one. They utilized the standard packet sequence representation, consisting of the sizes, times, and directions of the first 20 packets. The training pipeline featured two stages: a contrastive self-supervised phase, where augmented versions of the same sample were pulled together, followed by a supervised phase, where a classification head was trained on the extracted features. Wang et al. also experimented with a class-weighted sampler to achieve perfect class balancing in each training epoch but found no success with it. In contrast, our semi-balancing technique, detailed in Section III-D1, proved to be beneficial in our experiments. The differing outcomes may be due to the use of perfect class balancing, whereas we employed the λ_{sampler} parameter to control the strength of the balancing effect. We used the best classification accuracies reported by Wang et al. on the MIRAGE19 and MIRAGE22 datasets for SOTA comparison in Section V. Xie et al. [25] carefully designed three TCP-aware data augmentations that mimic real network dynamics, such as varying packet loss rates, retransmission timeouts, or the interplay between RTT and MTU in packet

buffering. Although their objective—to train encrypted traffic classifiers that are more robust under changing network conditions—aligns with our focus on general traffic representations, we chose not to incorporate data augmentations into our pipeline due to the increasing complexity of our proposed approach.

Finamore et al. [26] conducted a comprehensive evaluation of data augmentations of the FlowPic [27] representation, which is a 2D histogram that captures the evolution of packet sizes over time. The authors replicated an earlier study [28], reproducing most of the original results while also incorporating three additional datasets. The most relevant contribution for our work is the release of the *tcbench* open-source framework, which we use for transfer learning evaluation. Additionally, the best classification accuracies reported on the UCDAVIS19 and UTMOBILENET21 datasets are used for SOTA comparison.

III. EXPERIMENTAL SETUP

The overall domain recognition experimental setup is designed as a retrieval task for finding the most similar network flows, analogous to image retrieval tasks in computer vision. The SNI-based classes are divided into three disjoint sets: training, validation, and test. The training domain set is used to train a neural network, which serves as the embedding function that learns vector representations of network flows. This embedding function, denoted as Φ , is formalized in Eq. 1 and illustrated in Figure 1.

$$\Phi : \text{Flow} \rightarrow \mathbb{R}^d, d = \text{embedding size} \quad (1)$$

The validation domain set is used to measure performance during training, select the best model, and for finding the best configuration of hyperparameters. The test domain set is reserved for measuring and reporting the final metrics. The training, validation, and test domain sets are **disjoint**, so this setup pushes for strong generalization capabilities of the embedding function. It must learn patterns and extract traffic characteristics that remain useful for domains not seen during training. We based our experiments on the CESNET-QUIC22 [3] dataset, which is described in the next section.

A. CESNET-QUIC22 Dataset

The CESNET-QUIC22 [3] dataset includes four weeks of traffic captured at the monitoring points of the CESNET network, which is the national research and education network of the Czech Republic. The dataset consists of 153 million anonymized network flows enriched with various traffic features suitable for the classification of encrypted traffic. For this work, the relevant features are the packet sequences and SNI domains. The SNI domain is extracted from the Server Name Indication extension transmitted during the QUIC handshake. The packet sequences include packet sizes, inter-packet times, and packet direction of the first 30 packets. Prior work has shown that this packet sequence length is generally sufficient for accurate classification [1], [2]. For further details on the dataset and its collection process, including the software used,

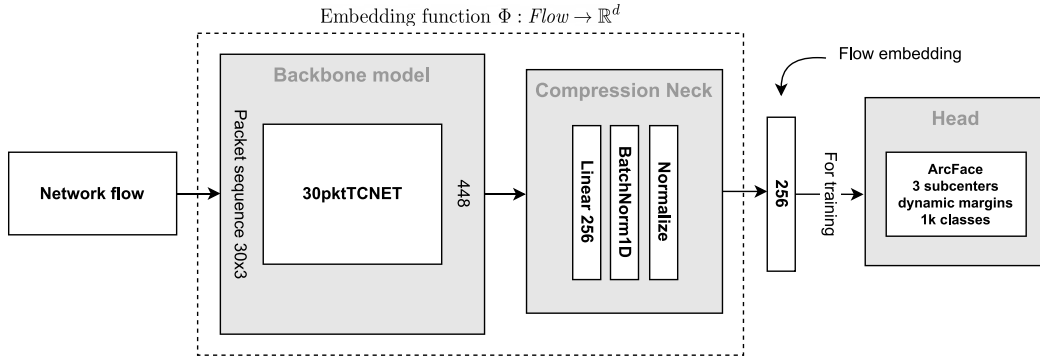


Fig. 1. A complete processing pipeline starting with network flows as input. The embedding function Φ , which is implemented as a neural network, maps flows into a 256-dimensional vector space. The visualized ArcFace head is used during training to optimize the neural network, which is composed of a backbone model and a compression neck.

sampling methods, flow export timeouts, and other relevant aspects, please refer to the original data article [3].

The dataset includes 102 service classes, where each *service* groups one or more domain names under a single label. However, since our objective is to predict individual domains, we do not use these predefined service labels. Instead, we utilized the exact SNI domains that are available in the dataset. Moreover, since the dataset contains general background traffic, the domains are not limited to the 102 service classes but represent all domains observed within the CESNET network.

In our previous experiments [2] with CESNET-QUIC22, we observed significant data drift in the traffic from the first two weeks of the dataset. In this work, we want to focus on evaluating the embedding function for domain recognition and transfer learning, without introducing extra complexities of the data drift. Therefore, we based our experiments on the third week of the dataset (W-2022-46) and used 33.7 million samples from this week for training, hyperparameter tuning, and the final evaluation of the proposed solution.

B. Experimental Pipeline

The following subsections describe the individual steps of the experimental setup, starting with domain preprocessing, followed by data preparation, training and validation, and finally model selection. An overview of the full pipeline is shown in Figure 2.

1) *Domain Preprocessing and Train/Val/Test Split*: The first step was to preprocess SNI domain names into class labels. We keep subdomains up to the fourth level and strip the rest (a.tile.openstreetmap.org is a fourth-level domain). Some domains contain a random string or parts related to locations or numbering. For example, the aforementioned openstreetmap domain has two “sister” domains [b,c].tile.openstreetmap.org. We decided to group such domains into a single class with the help of regexes. Another example would be europe-west1-gcp.api.snapchat.com and us-east4-gcp.api.snapchat.com, both remapped to a single class \$LOC-gcp.api.snapchat.com. In total,

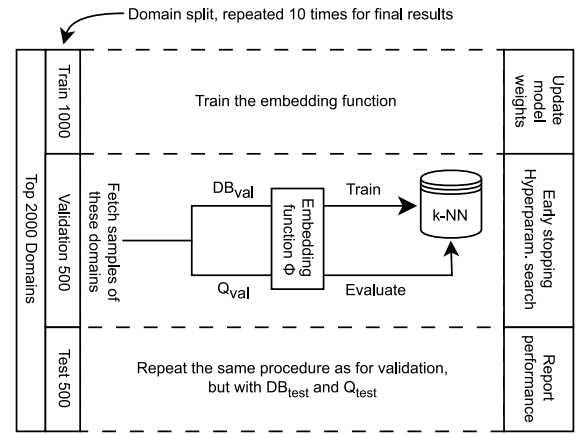


Fig. 2. An overview of the experimental setup, highlighting the purpose of the disjoint domain split along with the database and query preparation for validation and testing.

we created 40 regexes that remap thousands of domains with random parts into corresponding unified *domain classes*.

After this domain preprocessing, we selected the 2000 most frequent *domain classes* and divided them randomly into three subsets: 1000 for training, 500 for validation, and 500 for testing. These 2000 domains account for 99.38% of the total flows in the dataset’s third week.

2) *Database and Query Preparation*: Next, we prepare databases and query samples for validation and test domain sets. A database serves as a mini-training set for a k-NN classifier, which is then tested on query samples to measure performance. The exact same process is used for validation and test domains, and we will describe it for validation.

Validation samples (those having one of the 500 validation domains) are split into database and query parts. This split is random and stratified, meaning the class frequencies are preserved in both parts. We set the query part to contain one million samples and leave the rest for building the database. Out of these remaining samples, we randomly select one million to be included in the database. However, this second database sampling is not uniform but instead we soften the class imbalances. We set the weight of a sample belonging to class C to $N_C^{-\lambda_{db}}$, where N_C is the C class frequency

and $\lambda_{db} > 0$ a parameter controlling the strength of the balancing effect. We ended up using $\lambda_{db} = \frac{1}{2}$, meaning the weight formula is $\frac{1}{\sqrt{N_C}}$. This method for the selection of database samples prioritizes classes that are less frequent at the expense of the most frequent ones. To summarize, for both validation and test domain sets, we selected one million query samples (Q_{val} , Q_{test}) that follow the dataset's original class distribution, and we created a database (DB_{val} , DB_{test}) from one million samples with a more balanced class distribution.

3) *Training, Validation, and Ranking*: After preprocessing domains, splitting them into disjoint sets, and preparing databases and query samples, we can start training the Φ embedding function using the training domain set. The training loop runs for 30 epochs, measuring validation metrics each two epochs. The training loop is described in detail in Section III-D. The validation is performed as a similarity search in the embedding space. For each Q_{val} sample, we find the most similar DB_{val} samples, a process we refer to as database ranking. We limit the ranking to the closest 20 samples and compute several metrics that are described in Section IV-A to measure the quality of the embeddings. Each time, validation reuses the same DB_{val} and Q_{val} samples—what changes are the embeddings that are recomputed with model weights from the current epoch.

To compute distances in the embedding space, we use cosine similarity, which is a metric calculated as the dot product of L2-normalized vectors. When applied to L2-normalized vectors, cosine similarity produces the same ranking as the well-known Euclidean distance. To efficiently compute cosine similarities between all query samples and all database samples, we used the *faiss* library [29] specifically designed for efficient similarity search and clustering of vectors.

4) *Model Selection and Final Evaluation*: We chose the macro-average validation recall as our main objective, favoring models that perform well on all validation domains with equal importance (macro-averaging disregards sample count per domain). After the 30 training epochs, the model from the epoch with the best validation recall is saved and evaluated on the test domain set. This final evaluation is identical to the validation process but is using the test query samples Q_{test} and test database DB_{test} .

C. DL Architecture

We based the neural network architecture on our previous works on TLS and QUIC classification [1], [2], but due to substantial modifications, we describe it here in detail. The architecture is single-modal and designed to process fixed-sized packet sequences ($N = 30$) with the following features: packet sizes, inter-packet times (IPT), and directions. Network flows with fewer than N packets are padded with zeroes. The network is visualized in Figure 3. It follows the standard architecture of modern CNNs and consists of four main components: a stem, convolutional blocks, global pooling, and a feature refinement block.

1) *Stem*: The purpose of our network stem is to embed packet features into R -dimensional vectors to prepare them for subsequent processing with convolutions. This is achieved

using two PyTorch *Embedding*¹ layers—one for embedding of packet sizes and another for IPT. Each *Embedding* layer contains a learnable matrix with the shape *number of embeddings* $\times$ *embedding size*, where each row represents the embedding vector for a specific value. Packet sizes range from 0 to 1500, resulting in 1501 embeddings. For IPT, we first bin the values into 200 bins and then use the index of a bin as the input for the *Embedding* layer. Packet directions are one-hot encoded, which we found to be more effective than the traditional ± 1 encoding scheme. We set the embedding size to 20 for packet sizes and 10 for IPT. Thus, the stem outputs data in a shape (30×32) ,² where 30 corresponds to the packet sequence length, and 32 represents the combined embedding vector (20 for packet sizes, 10 for IPT, and 2 for directions).

The *Embedding* layers for packet sizes and IPTs were initialized using the Piecewise Linear Encoding (PLE) method proposed in [11], rather than the default random initialization. PLE creates initial embeddings structured as bins, where each bin corresponds to a segment of the feature's range (hence the name “piecewise”). Within each bin, linear relationships are preserved, maintaining the inherent ordering of numerical features. During training, the embeddings are optimized alongside the other model weights to adapt to the data. While embedding packet features before convolutional processing is not a novel concept—having been employed, for instance, by Nascita et al. [30], [31]—the PLE initialization technique has not yet been used in the TC domain.

2) *Convolutional Blocks*: The core processing in terms of feature extraction and parameter count is done with convolutions, implemented as four residual blocks adapted from the popular ResNet architecture. Specifically, we use Bottleneck Residual Block visualized in Figure 4, which was proposed in [32]. This block design reduces the number of parameters while preserving representational power. The term “bottleneck” refers to the temporary reduction of channels with a 1×1 convolution before the main convolution, followed by their restoration (or increase) afterward. Each block is defined by the following parameters: the main convolution kernel size k , the number of output channels C_{out} , the dropout rate, and the bottleneck ratio defining the reduction of channels for the main convolution (we use a common setting of $\frac{1}{4}$). The main convolution operation uses automatic padding to ensure that the spatial dimension (i.e., the length of packet sequences) remains unchanged. For the same purpose, we also use a stride of 1 in all convolutions, as we found that reducing the spatial dimension with a stride led to decreased performance. The four bottleneck blocks use different parameter settings: the number of output channels increases (192, 256, 384, 448), the kernel sizes decrease (7, 7, 5, 3), and the dropout rates progressively rise (0%, 10%, 20%, 30%). Overall, the convolutional blocks process packet embeddings of shape (30×32) and produce feature maps of shape (30×448) .

3) *Global Pooling*: The next component of the network is a global pooling operation, which aggregates each feature map along the spatial dimension (length 30) into a single

¹<https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>

²The batch size is omitted from all data shapes discussed in Section III-C.

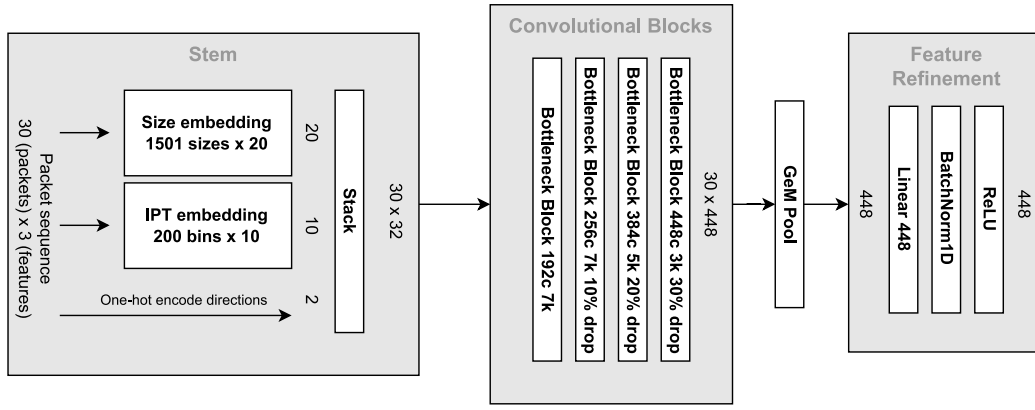


Fig. 3. The architecture of the 30pktTCNET backbone model consists of four main components: a stem, convolutional blocks, global pooling, and feature refinement. The main processing is done in the convolutional blocks, which include four Bottleneck Residual Blocks described in detail in Figure 4. Each block has a different configuration of the following parameters: the number of output channels (e.g., 256c), kernel size (e.g., 7k), and dropout rate.

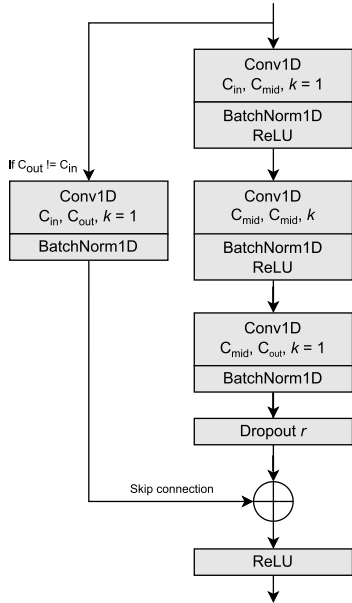


Fig. 4. The diagram of Bottleneck Residual Block. k : the kernel size of the main convolution, C_{out} : the number of output channels, r : the dropout rate. The number of channels of the main convolution C_{mid} is set to $\frac{C_{out}}{4}$. All convolutions use a stride of 1 and automatic padding to ensure that the spatial dimension is kept intact. Convolutions do not use biases.

scalar per channel, producing an output of size (1×448) . Common pooling methods include either averaging or taking the maximum of the values. We found maximum pooling to perform better than average pooling; however, our final choice was Generalized Mean Pooling (GeM) [10]. GeM includes a parameter p that enables interpolation³ between maximum pooling ($p \rightarrow \infty$) and average pooling ($p = 1$). The parameter p can be a fixed value or trained along with the other model weights. We initialized p to 3 and optimized it during training.

4) *Feature Refinement & Compression Neck*: The output of the GeM pooling is passed through a *feature refinement*

block—a simple sequence of Linear, BatchNorm, and ReLU layers. This *feature refinement block* preserves the shape of the features, resulting in an output size of 448.

Up to this point, the defined neural network architecture can be used for standard classification tasks; adding one extra Linear classification layer with a shape $(448 \times \text{number of classes})$ would do the job. However, our goal is to produce embeddings of network flows. Thus, as the final part of the network, we add a *compression neck* that is composed of a Linear layer with a shape (448×256) , BatchNorm, and a vector L2-normalization operation. The *compression neck* excludes a ReLU activation function on purpose, as its task is to compress features into the desired embedding size of 256 without introducing additional non-linear transformations.

We want to distinguish the backbone part—the stem, convolutional blocks, and the *feature refinement block*—from the entire neural network. We refer to this backbone as 30pktTCNET (visualized in Figure 3), while the complete model representing the Φ embedding function is denoted as 30pktTCNET_256 (visualized in Figure 1). With the final hyperparameter configuration, 30pktTCNET_256 has one million trainable parameters.

D. Training Loop

This section outlines the training loop that we used to optimize the neural network, describing the training sampler, loss function, optimizer, and learning rate (LR) scheduler.

1) *Training Sampler*: The training loop consists of 30 epochs. In each epoch, one million training samples are randomly selected for training. We used a modified random sampler, where the weights of samples of class C are set to $N_C^{-\lambda_{sampler}}$, with N_C being the C class frequency. We used the same value $\lambda_{sampler} = \frac{1}{2}$ as in the formula for selecting samples for the database. This approach softens class imbalances in the training set of each epoch, gives more focus on less frequent classes during training, and allows the model to learn traffic patterns from more diverse samples.

2) *ArcFace Loss Function*: In supervised metric learning, there are two main categories of loss functions used for training embedding models. The first consists of contrastive

³The exact GeM formula is $\left(\frac{1}{|X_c|} \sum_{x \in X_c} x^p \right)^{\frac{1}{p}}$, where X_c denotes a single channel from the output of the convolutional blocks.

approaches, such as Contrastive [33] or Triplet [34] loss, which pull together embeddings of samples with the same label and push apart those with different labels. These methods operate within each mini-batch, using local sample-to-sample comparisons. The other group of loss functions includes softmax-based losses with class centers and margin modifications, such as ArcFace [35] and CosFace [36]. These methods introduce class-specific centers and enforce angular or cosine margins to better separate classes. Samples are pulled toward their class centers based on global sample-to-class comparisons. ArcFace, along with its sub-center variant [37], represents the current state-of-the-art. While originally developed for face recognition, ArcFace has proven effective across various tasks like image retrieval and fine-grained classification. The following paragraph contains a short overview of how the ArcFace loss works. For a more technical and formal description, please refer to the original ArcFace paper [35].

Training a neural network with the ArcFace loss involves adding an “ArcFace head” (see Figure 1), which is detached and not used when the network is used for generating embeddings during validation and testing. The head contains a matrix of class centers, which are learnable during the training process. For computing the loss, both the embeddings and the class centers are normalized, which projects them onto a unit hypersphere. The ArcFace loss then calculates the angles between the flow embeddings and all the class centers. A fixed angular margin m is added to the angle corresponding to the correct class, which improves class separation (a larger angular gap between neighboring class centers). The cosine of these angles is then computed and scaled by a parameter s , producing the logits. These scaled logits are then passed to a cross-entropy loss function to calculate the final loss. By incorporating the angular margin, the ArcFace loss encourages the model to cluster embeddings of the same class closer together while increasing the angular separation between different classes, resulting in more discriminative embeddings.

3) *Sub-Center ArcFace With Dynamic Margins*: We experimented with an enhanced variant of ArcFace called sub-center ArcFace [37], which uses K sub-centers per class instead of a single center. During training, samples are pulled toward the nearest positive sub-center. This loss is better suited for handling intra-class variations, which are common in TC tasks—for instance, due to different API endpoints hosted behind a single SNI domain. Moreover, we used an ArcFace variant with dynamic margins, as introduced in [38] for tasks with extreme class imbalance. Each class uses a different angular margin, calculated as $m_C = a * N_C^{-\lambda_{margin}} + b$, where N_C is the C class frequency, a and b define the minimum and maximum margins, and $\lambda_{margin} > 0$ controls the rate of change in the margin. We used $\lambda = \frac{1}{4}$, with a and b set to produce margins in the range $[0.15, 0.25]$. Less frequent classes, which require greater separation in the embedding space for accurate classification, are given larger margins (up to 0.25) to widen their decision boundaries, whereas more frequent classes are assigned smaller margins (down to 0.15).

4) *Optimizer, LR Scheduler, Regularization, and Implementation Details*: The training loop was implemented

TABLE I
AN OVERVIEW OF THE HYPERPARAMETER SPACE

DL arch.	Stem: size of embeddings (packet sizes, IPT), IPT bins edges, directions encoding, PLE initialization
	Convolutions: block architecture, number of blocks, per-block channels, strides, dropout rates
	Pooling operation, activation and normalization functions
Training loop	Number of epochs, number of samples per epoch, semi-balanced sampling ($\lambda_{sampler}$)
	ArcFace loss function: scale, margins, K subcenters, λ_{margin} for dynamic margins
	Initial LR, AdamW optimizer params, LR scheduling, number of warm-up iterations, weight decay, weight initialization
	KoLeo regularization strength
DB	Database size, semi-balanced database sampling (λ_{db})
	Embedding size

in PyTorch. We used the AdamW optimizer with the default parameters, a batch size of 1024, and an initial learning rate of 0.0025. We used cosine learning rate decay, with a linear warm-up phase (from $\frac{0.0025}{3}$ to 0.0025) for the first 150 iterations. Weight decay of $0.0017 (\approx 10^{-2.75})$ was applied on all parameters except biases, BatchNorm affine parameters, packet size and IPT embedding matrices, and the GeM pooling p parameter. All weights used PyTorch’s default initialization, except for biases, which were set to zeros. We also use the KoLeo [39] regularization technique, which promotes a more uniform distribution of flow embeddings in the embedding space.

E. Hyperparameter Search

The hyperparameter search was conducted on a single domain split (described in Section III-B1), and the best-found parameters were reused for other domain splits. Our goal was to identify a configuration with the highest macro-average recall on the validation domain set. Due to the large hyperparameter space, a full grid search was infeasible; instead, we optimized and fixed subsets of hyperparameters step by step. The most important hyperparameters are summarized in Table I. In total, we used the MetaCentrum computing grid.⁴ to run more than 4000 trials, each taking approximately two hours.

IV. DOMAIN RECOGNITION RESULTS

This section presents experimental results for the domain recognition task. First, we describe performance metrics, introduce a simple baseline approach, and present the results. We conclude this section with ablations analyzing the influence of various components and parameters in the experimental setup.

A. Metrics

During validation and final testing, we perform database ranking to find the neighborhood of all query samples, as described in Section III-B3. The ranking relies on cosine similarity in the embedding space, where higher cosine similarity indicates more similar and closely related samples. To process a neighborhood into a domain prediction, we use three simple voting schemes: selecting the domain of the closest sample

⁴<https://www.metacentrum.cz/en/>

TABLE II
DOMAIN RECOGNITION RESULTS

Method	Acc	Recall	R-Q1	R-Q2	R-Q3	R-Q4
Baseline	71.44 (± 4.81)	37.86 (± 1.79)	51.91 (± 1.26)	38.95 (± 2.45)	32.77 (± 2.72)	27.81 (± 4.7)
Emb Φ	Top-1	94.83 (± 0.98)	79.35 (± 0.98)	89.63 (± 0.97)	81.36 (± 1.31)	75.99 (± 1.54)
	Maj-3	95.3	79.01	90.15	81.3	75.35
	Maj-5	95.54	78.37	90.44	80.94	74.38

Each value is an average over 100 runs (10 domain splits $\times$ 10 repetitions), except for the input-space baseline, which is averaged over 10 runs (10 domain splits $\times$ 1 repetition). Standard deviations for maj-3 and maj-5 are omitted to conserve space, as they do not differ from the top-1 values.

(top-1) or taking the majority domain among the three or five closest samples (maj-3, maj-5). In the case of ties (e.g., when all closest samples have different domains), the predicted domain is determined by the order of the neighboring samples. This approach is equivalent to using a k -NN classifier, where k corresponds to the size of the neighborhood.

For each voting scheme, we compute classification accuracy and macro-average recall. Among these metrics, we consider macro-average recall to be more important because it reflects overall performance across all domains, regardless of their frequencies. To better understand performance differences between frequent and infrequent domains, we also calculate macro-average recall for quartiles of domains sorted by frequency. For example, Q1 recall represents the macro-average recall for the top 25% most frequent domains, whereas Q4 recall corresponds to the bottom 25% least frequent domains.

B. Baseline Definition

It is good practice to compare deep learning models against simple baseline methods to better understand the contribution of more complex solutions. To this end, we devised an input-space baseline that uses raw packet sequences as embeddings. The experimental setup for this baseline is identical to that of the Φ embedding function, but flows are represented using the first 10 packet sizes, directions, and scaled inter-packet times. Because inter-packet times are generally less informative than packet sizes, they are clipped to a maximum of one second and scaled by a factor of $\frac{1}{10}$ to reduce their relative influence in distance computation. The baseline uses L1 distance for ranking and the top-1 voting scheme for making predictions.

C. Classification Performance

The final results for the domain recognition task were obtained as averages of 10 domain splits. Each domain split randomly divides the top 2000 domains into 1000 training domains, 500 validation domains, and 500 test domains. Additionally, for each domain split, we perform 10 repetitions, resulting in a total of 100 runs per reported value. Averaging over multiple domain splits ensures that the results are not biased for one specific set of domains. Results are presented in Table II, followed by a detailed discussion.

1) *Top-1 Performance*: The achieved top1-acc of 94.83% and recall⁵ of 79.35% are both surprising, and we consider them a success. This is remarkable given the challenging nature of our setup: the embedding function is evaluated on a set of domains disjoint from those used for training, and the task involves a large number of fine-grained classes. Measurements of recall across domain quartiles reveal interesting trends. For the most frequent Q1 domains, recall reaches 89.63%. Between Q1 and Q2, there is a notable recall drop of around 8%, and the subsequent gaps Q2 $\rightarrow$ Q3 and Q3 $\rightarrow$ Q4 are around 5% each. It is evident that less frequent classes are much harder to recognize, a phenomenon that is well-known and understandable for a wide range of ML tasks.

2) *Benefits of Using a Neighborhood With Maj-3 and Maj-5*: Considering a larger neighborhood of three or five samples introduces a trade-off between prioritizing Q1 domains and the rest. The Q1 recall improves for both maj-3 (90.15%) and maj-5 (90.44%) compared to top-1 (89.63%). However, the Q2–Q4 recalls for both maj-3 and maj-5 are lower than for top-1. This decrease can be attributed to sparse embeddings of less frequent domains, which have fewer database samples and, therefore, lack sufficient representation to “win” in the maj-3 and maj-5 voting schemes. Overall, using a larger neighborhood proves advantageous for the top 25% most frequent domains, while top-1 works better for the rest. The improved performance on the most frequent domains also explains that the classification accuracies of maj-3 (95.3%) and maj-5 (95.54%) exceed that of top-1 (94.83%), as frequent classes have a significant impact on micro-averaged metrics.

3) *Baseline Performance*: The input-space baseline was evaluated using the same experimental setup as the proposed Φ embedding function, enabling a direct performance comparison. The results show a significant 23.39% improvement in top1-acc of the proposed Φ embedding function over the baseline and even bigger improvements in Q1–Q4 recalls. This demonstrates that the baseline method is inadequate for addressing the domain recognition task within the given setup. However, our transfer learning experiments revealed that the input-space baseline can match SOTA performance on other TC datasets; see Section V-D3 for more results and related discussion.

D. Ablations

The purpose of ablation studies is to investigate a system’s performance by removing or modifying certain components to gain a better understanding of their contributions to the overall system. The following sections examine the role of certain hyperparameters and their effects on classification performance and ranking speed. All ablation experiments were conducted using one specific domain split (identical to that used in the hyperparameter search), and we report average results from 10 runs per configuration, if not stated otherwise.

1) *Packet Features - Direct Scalar Values, PLE Encoding, or Learnable Embedding Layer With PLE Initialization*: We evaluated the impact of our packet embedding scheme, referred to as Emb+PLE and detailed in Section III-C1, and present the results in Table III. Most related works use direct scalar

⁵All mentions of recall refer to top-1 recall, unless stated otherwise.

TABLE III
COMPARISON OF APPROACHES FOR ENCODING PACKET FEATURES

Method	Top1-Acc	Recall	R-Q1	R-Q2	R-Q3	R-Q4
Emb+PLE	95.75	79.29	90.38	79.59	74	73.18
PLE	95.67	78.49	90.19	78.64	73.03	72.11
Scalars	95.35	76.23	89.46	76.89	69.95	68.61
Emb+Rand	95.02	75.73	88.8	76.1	70.25	67.75

values, as did our previous architectures. In terms of recall, our embedding scheme (79.29%) shows a 3.06% improvement over the scalar approach (76.23%). The benefits are most prominent for less frequent domains, with a Q4 recall improvement of 4.57%. We also tested a variant denoted as PLE, which uses the initial embeddings created with PLE encoding without further optimization (i.e., *Embedding* weights are frozen). The results show PLE encoding accounts for most benefits, while the trainable *Embedding* layer adds a smaller incremental improvement. The gains in top1-acc are more modest, with Emb+PLE providing a 0.4% improvement over scalars.

Furthermore, we evaluated a variant denoted as Emb+Rand, which is similar to the Emb+PLE scheme but uses random initialization for the embedding matrices. It is likely that related works utilizing packet embeddings [30], [31] employ this variant, as no initialization method is specified and random initialization is the common default. Table III shows that Emb+Rand underperforms, further highlighting the strength of PLE initialization, which produces embeddings with inherent ordering and thereby facilitates the training process.

2) *Training Sampler - the Impact of the Balancing Parameter*: In each training epoch, one million network flows are sampled using a semi-balanced random sampler, where the λ_{sampler} parameter controls the balancing strength (see details in Section III-D1). When $\lambda_{\text{sampler}} = 0$, all samples are assigned equal weight, and the original imbalance is preserved (no balancing). When $\lambda_{\text{sampler}} = 1$, the sampler creates a per-domain distribution that is as uniform as possible (perfectly uniform distribution is unattainable as we use sampling without replacement).

Our expectation was that changing λ_{sampler} would provide a trade-off between focusing on more frequent with $\lambda_{\text{sampler}} = 0$ (higher top1-acc) or less frequent domains with $\lambda_{\text{sampler}} = 1$ (higher recall). However, it turned out that some degree of balancing has a positive impact even for the top1-acc metric. We ended up choosing $\lambda_{\text{sampler}} = \frac{1}{2}$, meaning the final weight formula is $\frac{1}{\sqrt{N_G}}$. Compared to a standard random sampler without balancing (which corresponds to $\lambda_{\text{sampler}} = 0$), this brings a 1.29% improvement in recall and a minuscule improvement of 0.13% in top1-acc. The impact on both metrics is showcased in Figure 5.

3) *Database - the Impact of the Balancing Parameter*: As described in Section III-B2, we also perform semi-balanced sampling for building the database. The weight formula is the same as for the epoch training sampler. When $\lambda_{\text{db}} = 0$, the original imbalance is preserved in the database. When $\lambda_{\text{db}} = 1$, the database has the per-domain distribution as uniform as possible (perfectly uniform distribution is unattainable as we use sampling without replacement). The graph

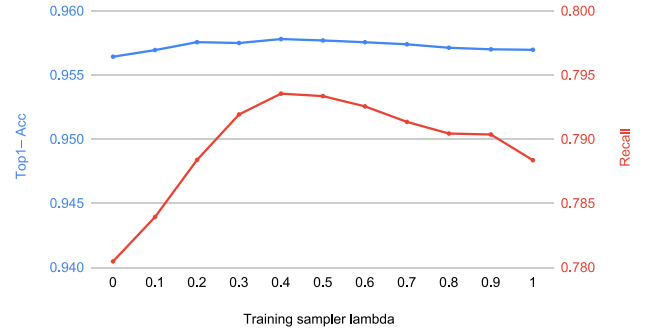


Fig. 5. The impact of the λ_{sampler} balancing parameter of training sampler. Both vertical axes of top1-acc and recall have a range of 2% to make the shapes of the lines comparable.

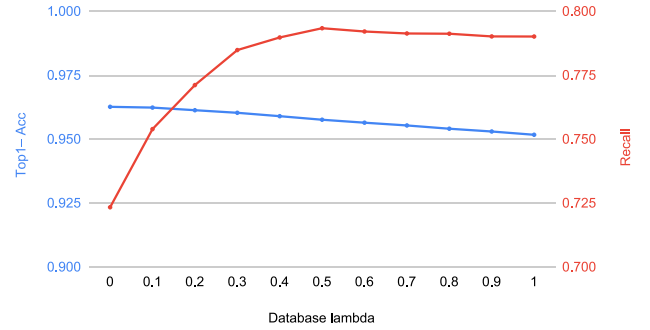


Fig. 6. The impact of the λ_{db} balancing parameter. Both vertical axes of top1-acc and recall have a range of 10% to make the shapes of the lines comparable.

investigating the impact of the λ_{db} parameter in Figure 6 shows a clear trade-off between decreasing top1-acc and increasing recall when λ_{db} moves from 0 to 1. We ended up choosing $\lambda_{\text{db}} = \frac{1}{2}$, which, compared to no balancing, brings a 7% improvement in recall at the expense of a 0.51% decrease in top1-acc. We believe this trade-off is worthwhile in most scenarios.

Combined effect: We also examined the combined effect of λ_{sampler} and λ_{db} . The results, presented in Table IV, highlight the performance gains compared to the case where neither the database nor the training set is balanced ($\lambda_{\text{sampler}} = 0$ and $\lambda_{\text{db}} = 0$).⁶ As expected, balancing has the greatest impact on Q4 recall, for which it provides a remarkable 13.47% improvement. The gains for Q3 (+11.04%) and Q4 (+8.39%) recalls are also impressive, especially given that the “cost” is merely a 0.4% decrease in top1-acc. Furthermore, the results indicate that database balancing and training set balancing operate independently, as their combined effect is approximately the sum of their individual contributions.

4) *Database - How Does the Number of Unique Domains Affect Performance?*: This section examines the sensitivity of the domain recognition approach to the number of unique domains we want to recognize. Previous work [40] demonstrated that TC tasks often become trivial when the number of classes is small. To explore this, we tested our approach with the number of unique domains ranging from 100 to 1000.

⁶Figure 5 shows the change between third and fourth rows of Table IV with $\lambda_{\text{db}} = \frac{1}{2}$ fixed, while Figure 6 shows the change between second and fourth rows with $\lambda_{\text{sampler}} = \frac{1}{2}$ fixed.

TABLE IV
THE COMBINED EFFECT OF BOTH DATABASE BALANCING λ_{db} AND $\lambda_{sampler}$

λ_{db}	$\lambda_{sampler}$	Top1-Acc	Recall	R-Q1	R-Q2	R-Q3	R-Q4
0.0	0.0	96.15	70.6	88.52	71.2	62.96	59.71
0.0	0.5	96.26 (+0.11)	72.29 (+1.69)	88.95 (+0.43)	72.70 (+1.5)	64.75 (+1.79)	62.74 (+3.03)
0.5	0.0	95.62 (-0.53)	78.00 (+7.4)	90.06 (+1.54)	78.42 (+7.22)	72.57 (+9.61)	70.95 (+11.24)
0.5	0.5	95.75 (-0.4)	79.29 (+8.69)	90.38 (+1.86)	79.59 (+8.39)	74.00 (+11.04)	73.18 (+13.47)

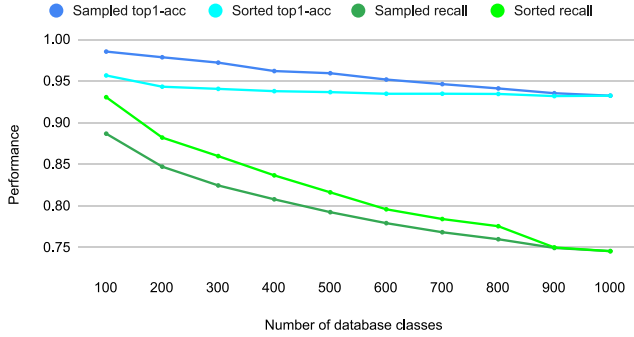


Fig. 7. Sensitivity of the proposed domain recognition approach to the number of unique domains.

To obtain up to 1000 domains, we first merge the validation and test domain sets.⁷ Then, we randomly select the desired number of domains, choose all samples of those domains, and split them into query and database parts. A database is created using the λ_{db} semi-balanced sampling. We measure the performance of a single trained model, which is reused for all domain counts and repetitions. For each domain count, we repeat this procedure 50 times and report the average. For the maximum of 1000 domains, we use all available validation and test domains. Additionally, we compare the described random domain sampling with a sorted approach, where we select the N (100, 200, ..., 1000) most frequent domains in each repetition. The results are presented in Figure 7. The top1-acc for random domain sampling ranges from 98.52% for 100 domains to 93.22% for 1000 domains, while the recall ranges from 88.65% to 74.54%. In contrast, the recall for the top 100 most frequent domains is as high as 93.04%, which is notable considering these 100 domains cover 85% of all dataset samples.

An interesting difference is observed between the domain selection methods: top1-acc is higher on random subsets of domains, while recall is better when using the top N domains. This is because when domains are sampled, some less frequent and harder-to-recognize domains are included. Since recall is macro-averaged, these harder domains have a larger impact on the overall metric. In contrast, for micro-averaged top1-acc, the inclusion of less frequent domains has minimal effect on the overall metric. Moreover, top1-acc is higher with sampled domains because there are fewer misclassifications in the region of the most frequent domains. Among these domains, there are a lot of similar ones that are prone to

⁷We acknowledge that reusing validation domains for testing deviates from our defined evaluation protocol; however, we did so only in this experiment to demonstrate performance across a wider range of classes.

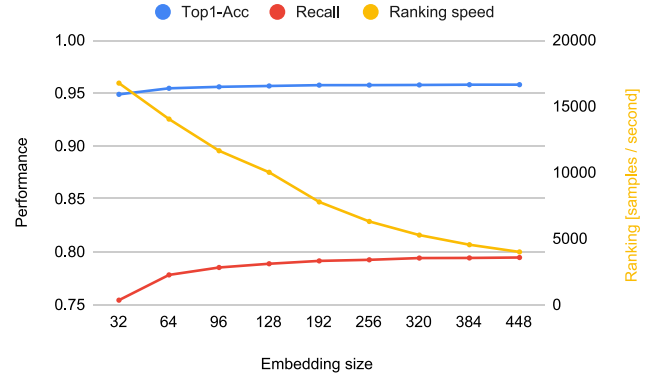


Fig. 8. The impact of the flow embedding size on domain recognition accuracy and ranking speed.

mismatch, and thus, top1-acc is increased when some of those are not selected in the given repetition.

5) *Embedding Size - Ranking Speed vs Recall Trade-Off*: The flow embedding size is a key parameter that influences the performance of the proposed domain recognition approach. Larger embeddings improve recognition accuracy but come at the cost of slower ranking speeds. To explore this trade-off, we ran experiments with embedding sizes ranging from 32 to 448. As with other ablations, we performed 10 repetitions per embedding size and report the average metrics. The results, summarized in Figure 8, show that reducing the embedding size has a limited impact on top1-acc (94.88% for size 32, 95.79% for size 448). However, recall is more sensitive, decreasing from 79.46% (size 448) to 75.43% (size 32).

We observed a clear inverse relationship between ranking speed and embedding size: the smallest embeddings achieved speeds of around 16.5k flows/s, while the largest slowed the ranking to 4k flows/s. In contrast, the speed of creating the embeddings with the neural network remained stable at around 33k flows/s, regardless of the used embedding size. Both tasks—creating embeddings and faiss⁸ ranking—were performed on an Nvidia Tesla T4 16GB GPU. Further performance-related discussion is provided in the final chapter.

6) *KoLeo Regularization*: We investigated the impact of KoLeo regularization, which uses a parameter λ to control its strength. Although KoLeo was originally designed [39] to improve embedding discretization—a step we do not perform—we observed that without this regularization, training diverges before completing the 30 training epochs. Table V

⁸The *faiss* library, which we use for database ranking, supports GPU indexes that offer a $5\times - 10\times$ performance boost compared to CPU implementations. <https://github.com/facebookresearch/faiss/wiki/Faiss-on-the-GPU>.

TABLE V
KoLeo REGULARIZATION IMPACT

KoLeo	Best epoch	Top1-Acc	Recall
Yes ($\lambda = 1$)	24.3	95.75	79.29
No ($\lambda = 0$)	22.6	95.68	78.97

compares the results with ($\lambda = 1$) and without ($\lambda = 0$) KoLeo. When KoLeo was used, the best validation performance was achieved later in training, indicating the neural network continued improving over more epochs. The later peak in validation performance suggests that KoLeo contributes to a more stable training process and enhances resistance to overfitting. However, the performance gains are modest: a 0.32% improvement in recall and a 0.07% in top1-acc.

V. TRANSFER TO DOWNSTREAM TRAFFIC CLASSIFICATION TASKS

To assess how well the proposed Φ embedding function pretrained on domain recognition generalizes, we evaluated it on ten established TC tasks and compared its performance to published SOTA results. We tested three transfer methods for each task: a k-NN classifier with fixed embeddings, a linear classifier with fixed embeddings (known as linear probing), and a linear classifier with fine-tuning of the embedding model. For each task, we also trained the same model from scratch in order to isolate the performance gains of the transfer learning methods. Furthermore, we measured the performance of the input-space baseline, which was defined in Section IV-B. Overall, we report six measurements for each downstream TC task: SOTA, input-space baseline, training from scratch, k-NN transfer method, linear probing, and embeddings fine-tuning. The following sections describe transfer methods, provide an overview of the used datasets, and discuss results.

A. Transfer Methods

For all transfer methods, we used “intermediate embeddings” of the Φ embedding function, taken after the global pooling and before the feature refinement block (see Figure 3). Features from deeper layers capture more general patterns and consistently offered better results for all tested methods.

1) *k-NN Transfer With Fixed Embeddings*: For the k-NN transfer method, we created embeddings for all samples of the given downstream task, trained a k-NN classifier on embeddings of the training set, and evaluated it on the test set. Predictions were based on the label of the closest training sample in the embedding space, and we did not use the λ_{db} semi-balancing technique.

2) *Linear Probing With Fixed Embeddings*: For linear probing, the setup was the same as for the k-NN method: we generated embeddings for the downstream task and trained a linear classifier on them. For smaller datasets (all except CESNET-TLS22 and AppClassNet), we used an exact solver from scikit-learn to fit the classifier, while for the large datasets, we used a single PyTorch Linear layer and implemented a simple training loop for it. Using the exact solver provided performance gains of around 1%, but it did not converge in a reasonable time on large datasets.

3) *Fine-Tuned Model*: The fine-tuning method uses a linear classification head and allows the entire 30pktTCNET backbone to be updated during training on the downstream task. Fine-tuning was performed over 50 epochs using the AdamW optimizer and a cosine learning rate schedule with a warm-up phase. All BatchNorm layers were kept in evaluation mode to preserve the batch statistics learned during pretraining. For each downstream task, we conducted a hyperparameter search over the learning rate, warm-up length, batch size, dropout rate of the classification head, and the pooling operation (allowing the original GeM pooling to be replaced with either max or average pooling). Hyperparameters were selected based on validation performance on the first data split.

If not addressed during fine-tuning, a neural network starts to forget its original capabilities and the general knowledge acquired during the pretraining task. To mitigate this, we implemented three fine-tuning techniques. (a) We use lower learning rates for deeper layers [41], which extract low-level traffic patterns analogous to edges and shapes in image processing. Along with other hyperparameters, we optimize a multiplicative factor, $LR_mult \in (0, 1]$, which scales the learning rate of the backbone’s residual blocks based on their depth, using the formula: $LR_{block} = LR \times LR_mult^{depth}$. (b) We applied L2 Starting Point (L2SP) regularization [42], a technique designed to mitigate catastrophic forgetting (i.e., the loss of knowledge from previous tasks). It adds a regularization term based on the L2 distance between the current model weights and the original pretrained weights (the starting point). This discourages large deviations from the pretrained weights, helps retain prior knowledge, and reduces overfitting on downstream tasks with limited training data. (c) A related but more recent method, called L2 Distance in Feature Space (LDIFS) [43], adds a regularization term based on the L2 distance between embeddings of the current batch created with the original (pretrained) model and the updated model. This encourages alignment between the original and fine-tuned feature spaces. While Mukhoti et al. [43] applied LDIFS to preserve intermediate features, we found that focusing on final features was sufficient for our use case. Both L2SP and LDIFS are controlled with regularization strength hyperparameters, where setting the value to zero disables the corresponding regularization. For downstream tasks with larger training sets (CESNET-TLS22, AppClassNet, MIRAGE19), neither regularization term was necessary. Three tasks benefited from a combination of both techniques, while the remaining tasks achieved the best fine-tuned performance using LDIFS alone.

B. Code and Pretrained Model Availability

We published the embedding model and its pretrained weights in the CESNET Models framework [44] under the name 30pktTCNET_256. The architecture code is available on GitHub.⁹ To select the best model (one specific set of weights), we chose the one with the highest sum of validation and test recalls on the domain recognition task, considering all training runs with the final hyperparameter configuration. Prior

⁹https://github.com/CESNET/cesnet-models/blob/main/cesnet_models/architectures/multimodal_cesnet_enhanced.py

to transfer to downstream tasks, we made a single modification to the model, targeting the packet size embedding technique implemented in the model stem.¹⁰ Thanks to the model's publication and the availability of open-source tools that provide access to the datasets, all transfer learning experiments presented in this section are reproducible. To support this, we have published our transfer learning codebase,¹¹ which enables replication of the results presented in Tables VI and VII.

C. Datasets

To evaluate transfer learning methods, we used seven additional datasets: ISCXVPN2016 [4], MIRAGE19 [5], MIRAGE22 [6], UTMOBILENET21 [7], UCDAVIS19 [8], CESNET-TLS22 [1], and AppClassNet [9], covering ten classification tasks in total. To streamline dataset handling, we used the *tcbench*¹² framework, which provides four datasets: MIRAGE19, MIRAGE22, UTMOBILENET21, and UCDAVIS19. These datasets are cleaned, pre-filtered when necessary, and include prepared train/validation/test splits. Overall, *tcbench* significantly simplified cross-dataset evaluation and saved us considerable time. CESNET-TLS22 was obtained via the CESNET DataZoo toolset [45], AppClassNet was downloaded from its official Figshare repository [46], and ISCXVPN2016 was kindly provided by Nascita et al. [31]. The following sections introduce each dataset and explain how it was used. We also provide related work used for SOTA comparison, with additional details about SOTA performance available in Appendix A. For all datasets, train/validation/test splits were used as follows: the training set was used to fit the k-NN classifier, perform linear probing, or fine-tune the model—depending on the selected transfer method. The validation set was used for hyperparameter search, and we report the average performance across all test splits.

ISCXVPN2016: A lab-generated traffic dataset that covers three classification tasks: encapsulation (VPN vs. nonVPN), traffic types, and applications. We used the preprocessed version provided by Nascita et al. [31], where broadcast flows and other noisy data were filtered out. We divided the dataset into ten stratified 60/20/20 train/validation/test splits. For SOTA comparison, we rely on the results reported by Nascita et al., who evaluated several models—some leveraging payload (e.g., the multi-modal network DISTILLER-Embeddings [31]), and others that do not (e.g., the 1D CNN over packet sequences by Rezaei and Liu [14]). Since we focus on TC solutions that do not use payload data, we primarily compare against models operating under this same constraint.

MIRAGE19: A well-known mobile traffic dataset from the MIRAGE dataset series. The traffic of this dataset is based on real users interactions with 20 Android applications. A private version that contains 40 applications exists but is not

part of *tcbench*. Aceto et al. [5] published this dataset in 2019, providing JSON files containing traffic capture experiments. The authors of *tcbench* processed the JSON files, removed background traffic, and discarded flows with fewer than 10 packets. This curation resulted in 64k samples, which were then used to create five 80/10/10 train/validation/test splits. For SOTA comparison, we rely on results reported by Wang et al. [24]. Even though the authors do not explicitly mention using *tcbench* data, their dataset curation, preprocessing steps, and splits are identical to those of *tcbench*. To be certain, we contacted the authors of both [24] and *tcbench*, who confirmed it. Therefore, their experiments and ours are based on identical data, making the results suitable for direct comparison.

MIRAGE22: Guarino et al. [6] introduced this mobile traffic dataset in 2022, focusing on video meeting apps like Zoom, Webex, and Teams. It includes traffic from nine Android applications. As the name suggests, MIRAGE22 comes from the same research group as MIRAGE19, and therefore the *tcbench* curation process is identical. For SOTA comparison, we rely on the best results reported by Wang et al. [24].

UTMOBILENET21: A mobile traffic dataset containing 17 Android applications, with user interactions emulated through the Android API. Heng et al. [7] published this dataset in 2021, providing packet information in CSV format. The authors of *tcbench* cleaned the data, assembled flows, filtered flows with less than 10 packets (9.5k samples remained), and prepared five 80/10/10 train/validation/test splits. For SOTA comparison, we rely on results reported by Finamore et al. [26], which is the paper that introduced the *tcbench* framework. We can thus be sure that experiments were performed on identical data and that the results are suitable for direct comparison.

UCDAVIS19: A small dataset published in 2019 [8] containing QUIC traffic of five Google services: Google Drive, Google Docs, Google Search, Google Music, and YouTube. It includes a *pretraining* partition with 6.5k samples and two test sets: *human* (83 samples) and *script* (150 samples). Although the small test sets limit the dataset's representativeness, we chose to include it in our evaluation as it is readily available in *tcbench*. We opted not to use the prepared splits, which consist of class-balanced subsets of the *pretraining* partition with only 100 samples per class. Instead, we create ten stratified 80/20 train/validation splits. For SOTA comparison, we rely on results reported by Finamore et al. [26].

CESNET-TLS22: A large TLS traffic dataset collected from the backbone lines of CESNET, the Czech national research and education network. The dataset spans two weeks and contains 141 million flows categorized into 191 Web service classes. Luxemburk and Čejka [1], the dataset authors, used the first week for training and the second week for testing—a time-based train-test split that we adopt in our experiments. Ten splits are created, each consisting of 1M training flows and 100k validation flows sampled from the first week, and 1M test flows from the second week. For SOTA comparison, we rely on results reported by Fauvel et al. [47].

AppClassNet: A large traffic dataset published in 2022 by Wang et al. [9] that contains 500 applications. It includes two official splits: one for the top 200 applications and another for

¹⁰In the model stem, the *Embedding* layer generates a learned vector representation for each packet size. However, some packet sizes (e.g., 1453–1471, 1473–1500) are never observed during training, leaving their representations untrained. To address this, we assign them the representation of the nearest observed packet sizes, except for packet sizes 1–19, which are given the representation of zero packet size.

¹¹<https://github.com/CESNET/tc-transfer>

¹²<https://github.com/tcbenchstack/tcbench>

TABLE VI
DOWNSTREAM PERFORMANCE OF THE THREE TRANSFER METHODS. DELTAS ARE COMPUTED RELATIVE TO TRAINING FROM SCRATCH ON EACH DOWNSTREAM TASK

Dataset			Downstream Task Performance [%]						
Name	Subtask	Classes	From Scratch	Linear Probing	k-NN Transfer	Fine-Tuned Model			
ISCXVPN2016	Application	15	75.71 (± 0.62)	72.81 ($\Delta -2.9$)	73.25 ($\Delta -2.46$)	77.44 (± 0.83)	($\Delta 1.73$)		
ISCXVPN2016	Traffic type	6	78.35 (± 0.95)	73.01 ($\Delta -5.34$)	75.57 ($\Delta -2.78$)	80.19 (± 0.92)	($\Delta 1.84$)		
ISCXVPN2016	Encapsulation	2	93.96 (± 0.45)	88.03 ($\Delta -5.93$)	91.83 ($\Delta -2.13$)	94.37 (± 0.45)	($\Delta 0.41$)		
MIRAGE19		20	82.49 (± 0.36)	72.51 ($\Delta -9.98$)	84.75 ($\Delta 2.26$)	85.31 (± 0.35)	($\Delta 2.82$)		
MIRAGE22		9	98.48 (± 0.15)	95.89 ($\Delta -2.59$)	98.19 ($\Delta -0.29$)	98.67 (± 0.27)	($\Delta 0.19$)		
UTMOBILENET21		17	85.3 (± 0.94)	86.43 ($\Delta 1.13$)	86.85 ($\Delta 1.55$)	88.9 (± 0.75)	($\Delta 3.6$)		
UCDAVIS19	Script test	5	98.73 (± 0.2)	98 ($\Delta -0.73$)	100 ($\Delta 1.27$)	98.13 (± 0.27)	($\Delta -0.6$)		
UCDAVIS19	Human test	5	78.07 (± 2.14)	87.47 ($\Delta 9.4$)	84.82 ($\Delta 6.75$)	89.64 (± 1.1)	($\Delta 11.57$)		
CESNET-TLS22		191	98.03 (± 0.03)	85.65 ($\Delta -12.38$)	96.44 ($\Delta -1.59$)	98.25 (± 0.02)	($\Delta 0.22$)		
AppClassNet		200	92.42 (± 0.02)	62.15 ($\Delta -30.27$)	81.12 ($\Delta -11.3$)	92.11 (± 0.03)	($\Delta -0.31$)		

Results are averages across ten splits. For *tcbench* datasets with five prepared splits, we used each split twice. Standard deviations of linear probing and k-NN transfer are omitted to conserve space, but they do not differ considerably from the values shown. All results are classification accuracies.

the remaining 300. We used the top-200 split as follows: the original training and validation sets were merged and re-split into ten stratified train/validation partitions (while keeping the test set unchanged). To ensure reasonable experiment runtimes, each split was then downsampled to 1M training flows, 100k validation flows, and 1M test flows. Before the release of AppClassNet, its packet sequences were transformed to protect business-sensitive information: feature amplitudes were modified, and sequences were partially shuffled. Wang et al. [9] evaluated several models on both the original and public (transformed) versions of the dataset; for SOTA comparison, we use the results reported on the public version.

D. Transfer Learning Results

This section presents transfer learning results obtained on ten downstream TC tasks. Section V-D1 and Table VI compare the three transfer methods with each other and with training from scratch, whereas Section V-D2 and Table VII benchmark the best-performing transfer method and the input-space baseline against the SOTA. Finally, Section V-D3 discusses the unexpectedly strong performance of the input-space baseline.

1) *Ranking of Transfer Methods*: The results in Table VI show a consistent performance ranking among the transfer methods: *linear probing* < *k-NN transfer* < *fine-tuned model*. This order holds across all downstream tasks except for the two UCDAVIS19 test sets. When we compare the fine-tuned model—the best transfer approach—with training from scratch, it performs better on all tasks except for AppClassNet and UCDAVIS19 *script* test, with an average improvement of 2.1%. Due to its feature transformation process, AppClassNet is a hard downstream task where training from scratch is expected to perform better. In the case of UCDAVIS19, which is the smallest dataset evaluated, the application of the LDIFS regularization technique led to a fine-tuned model that avoided overfitting compared to training from scratch. As a result, it performed much better on the *human* test (+11.57%, which comes from a different distribution than the training set) but underperformed on the *script* test (−0.6%, which shares the same distribution as the training set).

While fine-tuning is the overall best approach, we argue that the k-NN transfer method remains valuable in scenarios with limited downstream training data or when fine-tuning is not viable for other reasons. It outperforms training from scratch on four tasks and even surpasses SOTA on all tasks except CESNET-TLS22 and AppClassNet. The strong performance of k-NN transfer demonstrates that even the original, non-fine-tuned embeddings generalize well across diverse TC tasks.

2) *Surpassing SOTA*: We established that fine-tuning the entire model is the most effective transfer method. Table VII compares this approach to SOTA performance, showing that it surpasses it on nine of the ten downstream TC tasks, with an average improvement of 6.4%. In particular, strong gains exceeding 5% were achieved on the ISCXVPN2016, MIRAGE19, UTMObILENET21, and UCDAVIS19 datasets.

The performance on ISCXVPN2016 warrants a closer examination. For SOTA comparison, we used a 1D CNN model of Rezaei and Liu [14], which does not utilize payload as input. Nevertheless, our results are close to—or even surpass—the strongest SOTA method that *does* utilize payload data, namely DISTILLER-Embeddings from Nascita et al. [31] (results provided in Appendix A). The performance deltas of our fine-tuned model relative to DISTILLER-Embeddings are as follows: applications $\Delta -2.48\%$, traffic types $\Delta -1.52\%$, encapsulation $\Delta +1.36\%$. This comparison provides a better estimate of the actual value of including payload as model input on ISCXVPN2016 and shows that packet-sequence-processing and payload-processing models are much closer in performance than previously suggested.

3) *Surprising Performance of the Input-Space Baseline*: Among the transfer approach, the input-space baseline, and SOTA methods, we initially expected the input-space baseline to perform the worst due to its absolute simplicity. Surprisingly, however, it outperforms SOTA on four tasks (ISCXVPN2016 tasks and UTMObILENET21), delivers worse yet comparable performance on three (MIRAGE19, MIRAGE22, and UCDAVIS *script*), and falls significantly behind only on the remaining three (CESNET-TLS22, AppClassNet, and UCDAVIS *human*). To the best of our knowledge, this intriguing finding—that a

TABLE VII
COMPARISON OF THE BEST TRANSFER LEARNING METHOD (FINE-TUNED MODEL) WITH SOTA AND THE INPUT-SPACE BASELINE.
DELTAS ARE COMPUTED RELATIVE TO SOTA PERFORMANCE

Dataset			Downstream Task Performance [%]					State of the Art	
Name	Subtask	Classes	SOTA	Input Space		Best Transfer		Reference	Year
ISCXVPN2016	Application	15	63.92	70.91	(Δ 6.99)	77.44	(Δ 13.52)	Rezaei et al. [14]	2020
ISCXVPN2016	Traffic type	6	65.56	73.01	(Δ 7.45)	80.19	(Δ 14.63)	Rezaei et al. [14]	2020
ISCXVPN2016	Encapsulation	2	85.45	90.38	(Δ 4.93)	94.37	(Δ 8.92)	Rezaei et al. [14]	2020
MIRAGE19		20	80.06 [†]	79.93	(Δ -0.13)	85.3	(Δ 5.24)	Wang et al. [24]	2024
MIRAGE22		9	97.18 [†]	95.63	(Δ -1.55)	98.67	(Δ 1.49)	Wang et al. [24]	2024
UTMOBILENET21		17	81.91 [†]	83.86	(Δ 1.95)	88.73	(Δ 6.82)	Finamore et al. [26]	2023
UCDAVIS19	Script test	5	98.63	97.87	(Δ -0.76)	98.13	(Δ -0.5)	Finamore et al. [26]	2023
UCDAVIS19	Human test	5	80.45	72.77	(Δ -7.68)	89.64	(Δ 9.19)	Finamore et al. [26]	2023
CESNET-TLS22		191	97.2	90.95	(Δ -6.25)	98.25	(Δ 1.05)	Fauvel et al. [47]	2023
AppClassNet		200	88.3	76.2	(Δ -12.1)	92.11	(Δ 3.81)	Wang et al. [9]	2022

Results are averages across ten splits. For *tcbench* datasets with five prepared splits, we used each split twice.

[†] SOTA is reported in weighted F1-score; therefore, our results for these datasets are also reported using this metric. Nevertheless, we found the differences between the weighted F1-scores and classification accuracies are negligible: *Best Transfer* accuracies MIRAGE19 85.31%, MIRAGE22 98.67%, UTMOBILENET21 88.9%. All results for other datasets are classification accuracies.

simple k-NN classifier using features from the first 10 packets performs comparably to SOTA on several datasets—has not been reported in prior work and remains largely unexplored within the TC research domain. We emphasize that no dataset-specific modifications were made, such as adjusting the number of packets or the IPT scaling factor. If the baseline were tuned for each dataset, its performance could likely be improved even further.

We believe that the underlying cause is the high data redundancy in TC datasets. During dataset collection, it is highly probable—or almost certain for script-generated datasets—that multiple instances of the same network communication are captured, such as repeated API requests sent to the same server with identical TCP and TLS configurations. Consequently, TC datasets often contain numerous near-duplicate samples with the same label. When such a dataset is randomly split into training and test sets, these duplicates can end up in both sets. In this scenario, it is not surprising that a classifier relying on the closest training sample achieves high performance.

VI. LIMITATIONS

Nearest neighbors search: We want to address potential performance concerns related to our use of nearest neighbors search for the domain recognition task. In Section IV-D5, we measured the ranking speed across different embedding sizes. For instance, with an embedding size of 128, the ranking of a database containing one million samples achieves a speed of 10k samples per second. This speed is made possible thanks to *faiss*, which provides efficient methods for finding nearest neighbors and can even run on GPUs for faster processing. In this work, we opted to use the *IndexFlatIP* index that provides exact ranking results. However, *faiss* also offers other indexes that provide a trade-off between ranking speed and the precision of nearest neighbors search. Thus, if higher ranking speeds were needed, a natural solution would be to use an index with faster ranking, such as *IndexIVFFlat*¹³, at the cost of losing the guarantee of exact and exhaustive results.

Database construction: A further opportunity for improvement lies in the database construction process.

In this work, we used semi-balanced sampling to select one million samples for the database. We believe that a more informative strategy for choosing database samples—for instance, one guided by clustering—could reduce the database size while preserving domain recognition performance by prioritizing highly discriminative samples.

More research on the input-space baseline needed: Our motivation for designing a simple baseline and evaluating it under the same conditions as the Φ embedding function was to establish a reference point. The results, however, turned out to be far more intriguing than anticipated. In the cross-dataset evaluation, the baseline achieved performance quite close to SOTA, even surpassing it for the ISCXVPN2016 and UTMOBILENET21 datasets. Our hypothesis is that the data redundancy inherent in TC datasets, when combined with random splitting into training and test subsets, makes classification trivial for nearest neighbors search. As this observation may influence best practices for constructing and splitting TC datasets, further research is required to rigorously verify it.

VII. CONCLUSION

The main objective of this work was to design a universal embedding function suitable for a wide range of TC tasks. We first developed the Φ embedding function for the exact domain recognition task on the CESNET-QUIC22 dataset and then evaluated how well it generalizes to other TC datasets. To summarize the core components of the proposed embedding function: (a) the CNN-based feature extractor that embeds packet features before processing with ResNet-like blocks; (b) the ArcFace loss, which enhances class separation by pulling samples toward class centers while enforcing angular margins; and (c) a nearest neighbors classifier using cosine distance in the embedding space.

The domain recognition task is a significant challenge, particularly when monitoring networks with high volumes

¹³<https://github.com/facebookresearch/faiss/wiki/Faiss-indexes>

TABLE VIII
DOWNSTREAM TASK PERFORMANCE USING WEIGHTS TRAINED FROM SCRATCH ON SOURCE TASKS, REPLACING THE ORIGINAL DOMAIN RECOGNITION WEIGHTS. THE LAST ROW PRESENTS LINEAR PROBING RESULTS OF THE ORIGINAL MODEL FROM TABLE VI.
RESULTS ARE AVERAGES OF CLASSIFICATION ACCURACIES ACROSS FIVE SPLITS

Source Task	Downstream Task Performance [%]									
	ISCX Application	ISCX TrafficType	ISCX Encapsulation	MIRAGE19	MIRAGE22	UTMOBILENET21	UCDAVIS19 Script	UCDAVIS19 Human	CESNET-TLS22	AppClassNet
ISCX Application				46.14	75	71.29	91.87	50.6	46.84	45.98
ISCX TrafficType				46.29	79.06	72.3	90.8	54.22	47.05	39.62
ISCX Encapsulation				47.03	81.19	77.02	92.4	53.73	45.81	45.93
MIRAGE19	58.76	60.4	79.64		71.76	69.26	86.27	44.82	34.42	49.64
MIRAGE22	60.02	62.92	83.05	48.84		65.84	93.6	54.7	46.38	46.13
UTMOBILENET21	55.1	57.45	79.65	46.4	74.87		89.07	58.8	47.22	45.54
UCDAVIS19	40.64	49.47	74.95	39.52	72.02	70.06			19.72	36.34
CESNET-TLS22	68.81	69.67	85.29	66.17	94.22	82.58	98.27	64.1		60.15
AppClassNet	63.79	66.47	84.38	59.85	90.2	78.96	94	61.45	62.78	
QUIC Domain Recognition	72.81	73.01	88.03	72.51	95.89	86.43	98	87.47	85.65	62.15

of TLS and QUIC traffic protected by the ECH extension, which encrypts entire ClientHello messages and hides SNI domains from network operators. We tackled this task in a setup where domain names were disjoint across training, validation, and test sets, which forced the embedding function to learn traffic patterns that generalize to unseen domains. After an extensive hyperparameter search and tuning of our training pipeline and model architecture, we achieved a classification accuracy of 94.83% and a recall of 79.35%, which we consider a strong outcome considering the difficulty of the disjoint-class setup. The proposed architecture maintained a high throughput of 33k embedding inferences per second on a single Nvidia Tesla T4. We also conducted six ablations, each focusing on a specific component to assess its contribution to the overall solution. Notably, the combination of training and database semi-balancing samplers, along with the PLE initialization method for packet size and IPT *Embedding* layers, proved crucial for achieving high recall.

We then transferred the Φ embedding function to seven TC datasets: ISCXVPN2016, MIRAGE19, MIRAGE22, UTMOBILENET21, UCDAVIS19, CESNET-TLS22, and AppClassNet. The transfer learning approach proved to be highly successful, beating SOTA performance on nine of ten downstream TC tasks. Strong gains exceeding 5% were achieved on the ISCXVPN2016, MIRAGE19, UTMOBILENET21, and UCDAVIS19 datasets. Overall, the embedding function demonstrated strong generalization across all tested tasks. To our knowledge, no similar transfer learning achievements have been reported in the TC domain. We conclude that the domain recognition task, on which we developed and trained the embedding function, is well suited for the pretraining of TC models due to its complexity, large number of classes, and straightforward labeling process. An additional experiment presented in Appendix B evaluates individual TC tasks for pretraining, and the results indicate that domain recognition clearly outperforms the other tasks.

A. Future Directions

We conclude this paper by outlining future directions and discussing the advantages of leveraging a network flow's neighborhood in the embedding space. Producing a ranked list of the N closest samples—i.e., the most similar flows previously observed—offers flexibility in how the results are processed into final predictions. For example, server IP addresses or AS numbers could be used for additional post-filtering of the neighborhood. We see this as a reasonable direction for combining IP-related information with traffic shape characteristics. The ranking output also includes distance values that can support out-of-distribution detection: if all nearest samples exceed a predefined threshold, the prediction could be rejected or adjusted, for example by selecting the most common second-level domain in the neighborhood instead of the full domain prediction. Moreover, predictions based on the most similar samples are inherently more interpretable, as they are accompanied by concrete examples, even though the embeddings themselves are produced by a closed-box neural network. Finally, we believe that flow embeddings hold promise for other network monitoring tasks such as device profiling and identification.

APPENDIX A STATE-OF-THE-ART COMPARISON

For each dataset, we reference the prior work used for the SOTA comparison. We specify the evaluation metric and indicate the exact tables from which information on the best-performing classifiers was obtained. In some cases, measurements from multiple tables must be combined to provide the best achieved result reported in the cited work.

ISCXVPN2016, accuracy – Table III of Nascita et al. [31].

We primarily compare to a model that does not utilize payload as input, “*ID-CNN (PSQ)*”, which achieves 85.45% on the Encapsulation task, 65.56% on the Traffic Type task, and 63.92% on the Application task. In Section V-D2, we also compare to the

DISTILLER-Embeddings model, likewise reported in Table III of Nascita et al. [31]. This payload-processing model achieves 93.01% on the Encapsulation task, 81.71% on the Traffic Type task, and 79.92% on the Application task.

MIRAGE19, weighted F1-score – Table VII of Wang et al. [24]. Deltas from this table need to be added to the baseline performance of 75.43%. The best result is “MaskedStack ($p = 0.7$)” with 80.06% (75.43% + 4.63%).

MIRAGE22, weighted F1-score – Table VII of Wang et al. [24]. Deltas from this table need to be added to the baseline performance of 94.92%. The best result is “MaskedStack ($p = 0.3$)” with 97.18% (94.92% + 2.26%).

UTMOBILENET21, weighted F1-score – Table VIII of Finamore et al. [26]. The best result for the >10pkts version is “Time shift” with 81.91%.

UCDAVIS19, accuracy – Table VII of Finamore et al. [26], which reports results for an enlarged training set. The best result on the *human* test set is “SimCLR + fine-tuning” with 80.45%. For the *script* test set, the best result is “Packet loss” with 98.63%.

CESNET-TLS22, accuracy – Table V of Fauvel et al. [47]. The best result of 97.2% is achieved with the LEXNet architecture.

AppClassNet, accuracy – Table III of Wang et al. [9]. The best result of 88.3% on the public version of the dataset is achieved with random forest.

APPENDIX B

PRETRAINING $\times$ DOWNSTREAM TASK MATRIX

To further assess the strength of domain recognition as a pretraining task, we used individual TC tasks for pretraining instead of domain recognition. Specifically, we used models trained from scratch (those reported in the *From Scratch* column of Table VI). We then performed linear probing to evaluate how well these models transfer to other tasks. In other words, for each row in Table VIII, we repeated the linear probing experiment described in Section V-A, but replaced the original model weights obtained via domain recognition with weights trained from scratch on the source task.

We did not perform cross-transfer among the three ISCXPVN2016 tasks, as this would risk data snooping given our experimental setup with separate train/validation/test splits for these tasks. We chose linear probing for this experiment because it provides a straightforward interpretation of the results: it measures how linearly separable the features transferred across TC tasks are.

A. Results

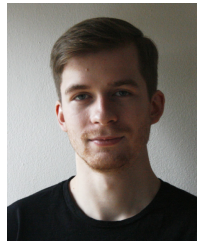
The results indicate that pretraining on other datasets produces features that do not generalize well. Larger datasets, such as CESNET-TLS22 and AppClassNet, achieve somewhat comparable performance but still lag behind domain recognition pretraining by approximately 5–10%. Smaller datasets perform poorly as pretraining tasks in this setup.

An open question is whether domain recognition itself constitutes an effective pretraining task, or whether the ArcFace-based training pipeline described in Section III is the primary driver behind the strong transfer performance. Evaluating and tuning the training pipeline for all source tasks is, however, beyond the scope of this paper. Nonetheless, we conclude that the combination of domain recognition—as a challenging task with hundreds of classes—and our optimized training pipeline produces features that are effective across all tested downstream tasks. Based on the preliminary experiments presented here, such effectiveness is difficult to achieve through pretraining on other source tasks.

REFERENCES

- [1] J. Luxemburk and T. Čejka, “Fine-grained TLS services classification with reject option,” *Comput. Netw.*, vol. 220, Jan. 2023, Art. no. 109467. [Online]. Available: <https://doi.org/10.1016/j.comnet.2022.109467>
- [2] J. Luxemburk, K. Hynek, and T. Čejka, “Encrypted traffic classification: The QUIC case,” in *Proc. 7th Netw. Traffic Meas. Anal. Conf. (TMA)*, 2023, pp. 1–10.
- [3] J. Luxemburk, K. Hynek, T. Čejka, A. Lukačovič, and P. Šiška, “CESNET-QUIC22: A large one-month QUIC network traffic dataset from backbone lines,” *Data Brief*, vol. 46, Feb. 2023, Art. no. 108888. [Online]. Available: <https://doi.org/10.1016/j.dib.2023.108888>
- [4] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, “Characterization of encrypted and VPN traffic using time-related features,” in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy*, 2016, pp. 407–414.
- [5] G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, “MIRAGE: Mobile-app traffic capture and ground-truth creation,” in *Proc. 4th Int. Conf. Comput. Commun. Security (ICCCS)*, 2019, pp. 1–8.
- [6] I. Guarino, G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, “Contextual counters and multimodal deep learning for activity-level traffic classification of mobile communication apps during COVID-19 pandemic,” *Comput. Netw.*, vol. 219, Dec. 2022, Art. no. 109452.
- [7] Y. Heng, V. Chandrasekhar, and J. G. Andrews, “UTMobileNetTraffic2021: A labeled public network traffic dataset,” *IEEE Netw. Lett.*, vol. 3, no. 3, pp. 156–160, Sep. 2021.
- [8] S. Rezaei and X. Liu, “How to achieve high classification accuracy with just a few labels: A semi-supervised approach using sampled packets,” 2020, *arXiv:1812.09761*.
- [9] C. Wang, A. Finamore, L. Yang, K. Fauvel, and D. Rossi, “AppClassNet: A commercial-grade dataset for application identification research,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 52, no. 3, pp. 19–27, Jul. 2022.
- [10] F. Radenović, G. Tolias, and O. Chum, “Fine-tuning CNN image retrieval with no human annotation,” 2018, *arXiv:1711.02512v2*.
- [11] Y. Gorishniy, I. Rubachev, and A. Babenko, “On embeddings for numerical features in tabular deep learning,” in *Proc. NeurIPS*, 2022, pp. 24991–25004.
- [12] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, “ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification,” in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [13] R. Zhao et al., “Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation,” in *Proc. 37th AAAI Conf. Artif. Intell. 35th Conf. Innov. Appl. Artif. Intell. 13th Symp. Educ. Adv. Artif. Intell.*, 2023, pp. 5420–5427. [Online]. Available: <https://doi.org/10.1609/aaai.v37i4.25674>
- [14] S. Rezaei and X. Liu, “Multitask learning for network traffic classification,” in *Proc. 29th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2020, pp. 1–9.
- [15] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, and A. Pescapé, “Classifying attack traffic in IoT environments via few-shot learning,” *J. Inf. Security Appl.*, vol. 83, Jun. 2024, Art. no. 103762. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214212624000656>
- [16] D. Di Monda, A. Montieri, V. Persico, P. Voria, M. De Ieso, and A. Pescapé, “Few-shot class-incremental learning for network intrusion detection systems,” *IEEE Open J. Commun. Soc.*, vol. 5, pp. 6736–6757, 2024.

- [17] D. D. Monda, G. Bovenzi, A. Montieri, V. Persico, and A. Pescapè, "IoT botnet-traffic classification using few-shot learning," in *Proc. IEEE Int. Conf. Big Data (BigData)*, 2023, pp. 3284–3293.
- [18] Y. Tian, Y. Wang, D. Krishnan, J. B. Tenenbaum, and P. Isola, "Rethinking few-shot image classification: A good embedding is all you need?" 2020, *arXiv:2003.11539*.
- [19] V. Tong et al., "Encrypted traffic classification through deep domain adaptation network with smooth characteristic function," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 331–343, Feb. 2025.
- [20] L. Gioacchini et al., "Cross-network embeddings transfer for traffic analysis," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 2686–2699, Jun. 2024.
- [21] L. Gioacchini, L. Vassio, M. Mellia, I. Drago, Z. B. Houidi, and D. Rossi, "DarkVec: automatic analysis of darknet traffic with word embeddings," in *Proc. 17th Int. Conf. Emerg. Netw. Exp. Technol.*, 2021, pp. 76–89. [Online]. Available: <https://doi.org/10.1145/3485983.3494863>
- [22] I. Guarino, C. Wang, A. Finamore, A. Pescapè, and D. Rossi, "Many or few samples? Comparing transfer, contrastive and meta-learning in encrypted traffic classification," in *Proc. 7th Netw. Traffic Meas. Anal. Conf. (TMA)*, 2023, pp. 1–10.
- [23] P. Khosla et al., "Supervised contrastive learning," 2020, *arXiv:2004.11362*.
- [24] C. Wang, A. Finamore, P. Michiardi, M. Gallo, and D. Rossi, "Data augmentation for traffic classification," in *Passive and Active Measurement*. Cham, Switzerland: Springer Nat., 2024, pp. 159–186. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-56249-5_7
- [25] R. Xie et al., "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-Aware traffic augmentation," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, Aug. 2023, pp. 625–642.
- [26] A. Finamore, C. Wang, J. Krolikowski, J. M. Navarro, F. Chen, and D. Rossi, "Replication: Contrastive learning and data augmentation in traffic classification using a FlowPic input representation," in *Proc. IMC*, 2023, pp. 36–51.
- [27] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [28] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-FlowPic augmentations," in *Proc. 22nd ACM Internet Meas. Conf.*, 2022, pp. 647–654. [Online]. Available: <https://doi.org/10.1145/3517745.3561436>
- [29] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [30] A. Nascita, A. Montieri, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapè, "XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4225–4246, Dec. 2021.
- [31] A. Nascita, A. Montieri, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapè, "Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.
- [32] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [33] S. Chopra, R. Hadsell, and Y. LeCun, "Learning a similarity metric discriminatively, with application to face verification," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, 2005, pp. 539–546.
- [34] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 815–823. [Online]. Available: <http://dx.doi.org/10.1109/CVPR.2015.7298682>
- [35] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 4685–4694.
- [36] H. Wang et al., "CosFace: Large margin cosine loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 5265–5274.
- [37] J. Deng, J. Guo, T. Liu, M. Gong, and S. Zafeiriou, "Sub-center ArcFace: Boosting face recognition by large-scale noisy Web faces," in *Proc. 16th Eur. Conf. Comput. Vis.*, 2020, pp. 741–757. [Online]. Available: https://doi.org/10.1007/978-3-030-58621-8_43
- [38] Q. Ha, B. Liu, F. Liu, and P. Liao, "Google landmark recognition 2020 competition third place solution," 2020, *arXiv:2010.05350*.
- [39] A. Sablayrolles, M. Douze, C. Schmid, and H. Jégou, "Spreading vectors for similarity search," 2019, *arXiv:1806.03198*.
- [40] L. Yang, A. Finamore, F. Jun, and D. Rossi, "Deep learning and zero-day traffic classification: Lessons learned from a commercial-grade dataset," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4103–4118, Dec. 2021. [Online]. Available: <https://doi.org/10.1109/tnsm.2021.3122940>
- [41] X. Dong et al., "CLIP itself is a strong fine-tuner: Achieving 85.7% and 88.0% top-1 accuracy with ViT-B and ViT-L on ImageNet," 2022, *arXiv:2212.06138*.
- [42] X. Li, Y. Grandvalet, and F. Davoine, "Explicit inductive bias for transfer learning with convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–12.
- [43] J. Mukhoti, Y. Gal, P. Torr, and P. K. Dokania, "Fine-tuning can cripple your foundation model: preserving features may be the solution," in *Proc. Trans. Mach. Learn. Res.*, 2024, pp. 1–26. [Online]. Available: <https://openreview.net/forum?id=kfhoeZCeW7>
- [44] J. Luxemburk and K. Hynek, "Towards reusable models in traffic classification," in *Proc. 8th Netw. Traffic Meas. Anal. Conf. (TMA)*, 2024, pp. 1–4.
- [45] J. Luxemburk and K. Hynek, "DataZoo: Streamlining traffic classification experiments," in *Proc. Explainable Safety Bounded, Fidelity, Mach. Learn. Netw.*, 2023, pp. 3–7. [Online]. Available: <https://doi.org/10.1145/3630050.3630176>
- [46] D. Rossi, "AppClassNet—A commercial-grade dataset for application identification research," Aug. 2022. [Online]. Available: https://figshare.com/articles/dataset/AppClassNet_-_A_commercial-grade_dataset_for_application_identification_research/20375580
- [47] K. Fauvel, F. Chen, and D. Rossi, "A lightweight, efficient and explainable-by-design convolutional neural network for Internet traffic classification," in *Proc. KDD*, 2023, pp. 4013–4023.



Jan Luxemburk He is currently pursuing the Ph.D. degree with the Faculty of Information Technology, Czech Technical University in Prague. He is a Researcher with CESNET. His research focuses on applying deep learning to network traffic monitoring, particularly on classifying encrypted TLS and QUIC traffic using packet-level metadata sequences. He contributed to the design of large-scale traffic collection infrastructure at CESNET and curated several publicly available datasets, including CESNET-TLS22 and CESNET-QUIC22.



Karel Hynek received the Ph.D. degree from the Faculty of Information Technology, Czech Technical University in Prague. He is a Researcher with CESNET, the Czech national research and education network. His expertise lies in network security, with an emphasis on high-speed monitoring and ISP-scale protection systems. His research outputs, such as traffic classifiers, detectors, and data exporters, are actively deployed to support and protect a production ISP monitoring infrastructure.



Richard Plný is currently pursuing the Ph.D. degree with the Faculty of Information Technology, Czech Technical University in Prague. His research focuses on explainable network traffic classification based on heterogeneous methods and data fusion. He is also a Researcher with CESNET, where he works on threat detection in large ISP-level networks, including the identification of cryptomining activity.



Tomáš Čejka is an Associate Professor with the Faculty of Information Technology, Czech Technical University in Prague. His research is centered around high-speed network traffic monitoring and analysis. He founded the NETMON Laboratory, where he supervises bachelor's, master's, and doctoral students. He also serves as the Head of a Research Department with CESNET, the operator of the Czech national research and education network.